

Модуль 11. Асинхронная работа с Kafka: клиент `aiokafka`

11.1. Зачем асинхронный клиент: не блокируйте API

В Модуле 10 мы отправляли сообщения в Kafka через консольные утилиты (`kafka-console-producer.sh`). Это было полезно для понимания, но в реальном приложении вы не вызываете shell-команды из Python. Вы используете **клиентскую библиотеку**, которая говорит с Kafka напрямую по сетевому протоколу.

Существует два основных клиента для Python:

- `kafka-python` — синхронный (блокирующий).
- `aiokafka` — асинхронный (неблокирующий, на базе `asyncio`).

11.1.1. Проблема блокировки

Представьте, что внутри endpoint FastAPI вы отправляете событие в Kafka синхронным клиентом:

```
In [1]: @app.post("/predict")
def predict(data: str):
    # Синхронная отправка: программа стоит и ждёт ответа от Kafka
    producer.send("predictions", {"data": data}) # может занять 50-200 мс
    return {"status": "ok"}
```

Что происходит: Пока сообщение летит по сети к Kafka-брокеру, обрабатывается там и возвращается подтверждение — ваш сервер **стоит**. Он не может обслуживать других клиентов. Если у вас 100 запросов в секунду, и каждый ждёт 100 мс — ваш API захлебнётся.

Аналогия: Вы — бариста в кофейне. Клиент заказал кофе. Вы стоите у кофемашины и **не отходите**, пока она не сварит кофе. Очередь за вами растёт. Вы не принимаете новые заказы, не выдаёте готовые напитки. Вы просто ждёте.

11.1.2. Решение: асинхронность

Асинхронный клиент (`aiokafka`) позволяет сказать: «Отправь сообщение в Kafka, а я пока займусь другими клиентами. Когда Kafka ответит — позови меня.»

```
In [2]: @app.post("/predict")
async def predict(data: str):
    # Неплокирующая отправка: сервер сразу освобождается
```

```
await producer.send("predictions", {"data": data})
return {"status": "ok"}
```

Аналогия: Тот же бариста, но теперь он нажимает кнопку на кофемашине, записывает номер заказа на стикер, клеит на стакан и **сразу** поворачивается к следующему клиенту. Когда кофемашина пикнет — он знает, что кофе готов, и выдаёт его. Время ожидания не тратится впустую.

11.1.3. Что такое `async` / `await` (кратко, но достаточно)

В Python есть механизм `asyncio`. Ключевые слова:

- `async def` — объявляет функцию, которая может «отпускать» управление.
- `await` — говорит: «здесь мне нужно дождаться результата, но пока жду — пусть другие задачи поработают».

Важно для новичка: Вам не нужно становиться экспертом по `asyncio`. Достаточно правила: если функция объявлена как `async def` — вызывайте её с `await`. Если библиотека называется `aiokafka` — она асинхронная, и все её методы вызываются с `await`.

11.2. Установка и подготовка

Шаг 1. Установка библиотеки

```
In [3]: pip install aiokafka
```

Если вы работаете в виртуальном окружении из предыдущих модулей:

```
In [4]: source ~/docker-module10/venv/bin/activate # или где у вас venv
pip install aiokafka fastapi uvicorn
```

Шаг 2. Убедитесь, что Kafka запущена

Если контейнер из Модуля 10 остановлен:

```
In [5]: cd ~/docker-module10
docker compose up -d
```

Проверьте:

```
In [6]: docker logs kafka_broker | grep "Kafka Server started"
```

11.3. Асинхронный Producer: публикуем события из Python

11.3.1. Создание Producer'a

Создайте файл `kafka_producer.py` :

```
In [7]: import asyncio
import json
from aiokafka import AIOKafkaProducer

# Адрес Kafka (у нас один брокер на localhost:9092)
KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC_NAME = "ml.predictions"

async def send_prediction_event(model_id: str, accuracy: float, dataset: str):
    """
    Отправляет событие об обучении модели в Kafka.
    """
    # Создаём Producer
    # value_serializer говорит: "перед отправкой преврати dict в JSON-байты"
    producer = AIOKafkaProducer(
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    )

    # Подключаемся к Kafka (устанавливаем TCP-соединение)
    await producer.start()

    try:
        # Формируем сообщение
        event = {
            "event_type": "model_trained",
            "model_id": model_id,
            "accuracy": accuracy,
            "dataset": dataset,
        }

        print(f"[PRODUCER] Отправляю событие: {event}")
```

```

# Отправляем сообщение в топик
# await означает: "отправь, и когда Kafka подтвердит получение — продолжи"
result = await producer.send_and_wait(TOPIC_NAME, event)

# result содержит метаданные: в какую партицию попало сообщение и какой of
fset

print(f"[PRODUCER] TRUE Сообщение доставлено!")
print(f"        Топик: {result.topic}")
print(f"        Партиция: {result.partition}")
print(f"        Offset: {result.offset}")

except Exception as e:
    print(f"[PRODUCER] FALSE Ошибка отправки: {e}")

finally:
    # Всегда закрываем соединение при выходе
    await producer.stop()

async def main():
    # Отправляем несколько событий
    await send_prediction_event("model_xgboost_v1", 0.9432, "customers_2024.csv")
    await send_prediction_event("model_nn_v2", 0.9121, "customers_2024.csv")
    await send_prediction_event("model_linear_v1", 0.8876, "sales_q2.csv")

if __name__ == "__main__":
    asyncio.run(main())

```

Разбор ключевых моментов:

`AIOKafkaProducer(...)`

Создаёт объект-продюсер. Пока он не подключён к Kafka — это просто «пустая оболочка».

`value_serializer=lambda v: json.dumps(v).encode("utf-8")`

Kafka хранит **байты**, не dict и не JSON-строки. Эта функция автоматически превращает ваш Python-словарь в JSON, а JSON — в байты (`b'{"key": "value"}'`). Без этого пришлось бы вручную кодировать каждое сообщение.

`await producer.start()`

Устанавливает TCP-соединение с брокером, запрашивает метаданные (какие топики есть, кто лидер). Это асинхронная операция — соединение устанавливается не мгновенно.

```
await producer.send_and_wait(TOPIC_NAME, event)
```

Главный метод. Он:

1. Сериализует сообщение.
2. Определяет партицию (round-robin, так как нет ключа).
3. Отправляет сообщение брокеру.
4. **Ждёт подтверждения** (ack) от брокера.
5. Возвращает `RecordMetadata` — информацию о том, куда именно попало сообщение.

Почему `send_and_wait` , а не просто `send` ?

- `send()` — отправляет сообщение во внутренний буфер Producer'a и сразу возвращает `asyncio.Future` . Быстро, но вы не знаете, доставлено ли сообщение.
- `send_and_wait()` — отправляет и **ждёт подтверждения** от брокера. Надёжнее, но чуть медленнее.

Для обучения используйте `send_and_wait` . В production с высокой нагрузкой используют `send()` с пакетированием (batching).

```
await producer.stop()
```

Закрывает соединение, сбрасывает буферы. Всегда вызывайте в `finally` , иначе сообщения, которые ещё не улетели, потеряются.

11.3.2. Запуск и проверка

```
In [8]: cd ~/docker-module10 # или ~/docker-module11, если создали новую
python kafka_producer.py
```

Вывод:

```
In [9]: [PRODUCER] Отправляю событие: {'event_type': 'model_trained', ...}
[PRODUCER] TRUE Сообщение доставлено!
        Топик: ml.predictions
        Партиция: 1
        Offset: 0
        ...
```

Проверьте, что сообщения появились в Kafka, через консольного Consumer'a из Модуля 10:

```
In [10]: docker exec -it kafka_broker \
         kafka-console-consumer.sh \
         --topic ml.predictions \
         --from-beginning \
         --bootstrap-server localhost:9092
```

Вы увидите ваши JSON-сообщения.

11.4. Асинхронный Consumer: читаем поток событий

11.4.1. Создание standalone Consumer'a

Создайте `kafka_consumer.py` :

```
In [11]: import asyncio
import json
from aiokafka import AIOKafkaConsumer

KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC_NAME = "ml.predictions"

async def consume_events():
    """
    Читает события из Kafka и обрабатывает их.
    """
    # Создаём Consumer
    # value_deserializer обратен serializer'у: байты -> JSON -> dict
    consumer = AIOKafkaConsumer(
        TOPIC_NAME,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id="ml_analytics_group", # имя группы потребителей
        auto_offset_reset="earliest", # если группа новая — читать с начала
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    )

    # Подключаемся
    await consumer.start()

    try:
        print("[CONSUMER] Ожидаю сообщения из топика 'ml.predictions'...")
        print("[CONSUMER] Нажмите Ctrl+C для остановки.\n")
```

```

# Бесконечный цикл чтения
# async for – асинхронный итератор. Он ждёт сообщения, не блокируя другие
задачи.

async for msg in consumer:
    print(f"[CONSUMER] Получено сообщение:")
    print(f"            Топик: {msg.topic}")
    print(f"            Партиция: {msg.partition}")
    print(f"            Offset: {msg.offset}")
    print(f"            Ключ: {msg.key}")
    print(f"            Значение: {msg.value}")
    print(f"            Время: {msg.timestamp}")
    print("-" * 50)

    # ИМИТАЦИЯ ОБРАБОТКИ
    # Например, записать в базу данных, обновить дашборд, отправить алерт
    await process_event(msg.value)

except asyncio.CancelledError:
    print("\n[CONSUMER] Получен сигнал остановки.")
finally:
    await consumer.stop()
    print("[CONSUMER] Соединение закрыто.")

async def process_event(event: dict):
    """
    Обработывает одно событие.
    В реальности здесь могла бы быть запись в БД, вызов API и т.д.
    """
    event_type = event.get("event_type")
    model_id = event.get("model_id")
    accuracy = event.get("accuracy")

    # Имитация долгой обработки
    await asyncio.sleep(1)

    if accuracy and accuracy > 0.93:
        print(f"            🏆 Отличная модель! {model_id} с accuracy {accuracy}")
    else:
        print(f"            📊 Модель {model_id} обработана.")

if __name__ == "__main__":
    try:
        asyncio.run(consume_events())

```

```
except KeyboardInterrupt:
    print("\n[MAIN] Программа завершена пользователем.")
```

Разбор ключевых моментов:

`AIOKafkaConsumer(...)`

- Первый аргумент — имя топика (или список топиков).
- `group_id` — **критически важно**. Это имя Consumer Group. Kafka запомнит, до какого offset'a дочитала эта группа. Если вы перезапустите скрипт — он продолжит с места остановки, а не с начала.
- `auto_offset_reset="earliest"` — если группа никогда не читала этот топик (первый запуск), начать с самого старого сообщения (`earliest`). Альтернатива: `"latest"` — начать с новых, игнорируя старые.

`async for msg in consumer:`

Это «сердце» Consumer'a. `consumer` — это асинхронный итератор. Он:

1. Запрашивает сообщения у Kafka.
2. Если сообщений нет — «засыпает», отдавая управление другим задачам.
3. Как только сообщение приходит — просыпается и отдаёт его в цикл.

`msg.value` — уже десериализованный Python-dict (благодаря `value_deserializer`).

`msg.offset` — позиция сообщения в партиции. Полезно для отладки.

11.4.2. Запуск и наблюдение

```
In [12]: # Терминал 1: запускаем Consumer
python kafka_consumer.py

# Терминал 2: запускаем Producer из предыдущего раздела
python kafka_producer.py
```

Наблюдайте:

- Consumer получает сообщения по мере их отправки.
- После обработки каждого сообщения Consumer автоматически отправляет **commit** (подтверждение offset'a) в Kafka. Это происходит в фоне благодаря `enable_auto_commit=True` (по умолчанию).

- Если вы остановите Consumer (`Ctrl+C`), запустите снова — он **пропустит** уже обработанные сообщения и начнёт с новых. Потому что Kafka запомнила offset группы `ml_analytics_group` .

11.4.3. Ручное управление offset'ами (commit)

По умолчанию `aiokafka` автоматически коммитит offset'ы каждые 5 секунд. Но в production часто нужен **ручной контроль**: вы коммитите offset только после того, как уверены, что сообщение полностью обработано (например, записано в БД).

```
In [13]: consumer = AIOKafkaConsumer(
    TOPIC_NAME,
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
    group_id="ml_analytics_group",
    auto_offset_reset="earliest",
    value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    enable_auto_commit=False, # <- отключаем автокоммит
)

# ...

async for msg in consumer:
    try:
        await process_event(msg.value)

        # Явно говорим Kafka: "Я успешно обработал сообщение до offset X"
        await consumer.commit()

    except Exception as e:
        print(f"Ошибка обработки: {e}")
        # Не делаем commit! Сообщение будет прочитано снова при перезапуске.
        # В production здесь может быть логика retry или dead letter queue.
```

Зачем это нужно?

Представьте, что `process_event` записывает данные в PostgreSQL. Если вы закомитили offset **до** записи в БД, а потом запись в БД упала — сообщение потеряно (вы его больше не прочитаете, но и в БД его нет). Ручной commit гарантирует атомарность: либо и в БД, и в Kafka, либо нигде.

11.5. Интеграция aiokafka с FastAPI

Теперь соберём всё вместе. Создадим FastAPI-приложение, которое:

1. При старте подключается к Kafka как Producer.
2. Имеет endpoint, который публикует события.
3. На фоне (параллельно с API) запускает Consumer, который читает события и логирует прогресс.

Шаг 1. Структура проекта

Создайте папку `~/docker-module11` со следующими файлами:

```
In [14]: docker-module11/
          ├── main.py          # FastAPI приложение + Kafka интеграция
          ├── requirements.txt
          └── docker-compose.yml # (опционально, если хотим всё в Docker)
```

Шаг 2. requirements.txt

```
In [15]: fastapi==0.111.0
          uvicorn[standard]==0.30.0
          aiokafka==0.11.0
```

Шаг 3. Полноценное приложение `main.py`

```
In [16]: import asyncio
          import json
          import logging
          from contextlib import asynccontextmanager

          from fastapi import FastAPI, HTTPException
          from pydantic import BaseModel
          from aiokafka import AIOKafkaProducer, AIOKafkaConsumer

          # --- Настройка логирования ---
          logging.basicConfig(level=logging.INFO)
          logger = logging.getLogger(__name__)

          # --- Конфигурация ---
          KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
          TOPIC_NAME = "app.events"

          # Глобальные переменные для Producer'а и задачи Consumer'а
          producer: AIOKafkaProducer | None = None
```

```

consumer_task: asyncio.Task | None = None

# =====
# МОДЕЛИ ДАННЫХ
# =====

class EventRequest(BaseModel):
    user_id: int
    action: str
    payload: dict | None = None

class EventResponse(BaseModel):
    status: str
    topic: str
    partition: int
    offset: int

# =====
# LIFESPAN: что происходит при старте и остановке приложения
# =====

@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    lifespan заменяет старые @app.on_event("startup")/@app.on_event("shutdown").
    Он гарантирует, что ресурсы (Kafka) корректно инициализируются и закрываются.
    """
    global producer, consumer_task

    logger.info("[LIFESPAN] Запуск приложения...")

    # 1. Создаём и стартуем Producer
    producer = AIOKafkaProducer(
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    )
    await producer.start()
    logger.info("[LIFESPAN] Kafka Producer подключен.")

    # 2. Запускаем Consumer как фоновую задачу
    consumer_task = asyncio.create_task(kafka_consumer_loop())
    logger.info("[LIFESPAN] Kafka Consumer запущен на фоне.")

```

```

yield # <- здесь приложение работает и обрабатывает запросы

# 3. При остановке приложения
logger.info("[LIFESPAN] Остановка приложения...")

if consumer_task:
    consumer_task.cancel()
    try:
        await consumer_task
    except asyncio.CancelledError:
        pass

if producer:
    await producer.stop()

logger.info("[LIFESPAN] Kafka соединения закрыты.")

# =====
# СОЗДАНИЕ ПРИЛОЖЕНИЯ
# =====

app = FastAPI(
    title="FastAPI + Kafka (aiokafka)",
    description="API публикует события в Kafka, фоновый Consumer их обрабатывает.",
    lifespan=lifespan,
)

# =====
# CONSUMER: фоновая корутина
# =====

async def kafka_consumer_loop():
    """
    Эта функция работает в фоне на протяжении всей жизни приложения.
    Она читает сообщения из Kafka и обрабатывает их.
    """
    consumer = AIOKafkaConsumer(
        TOPIC_NAME,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id="fastapi_app_consumers",
        auto_offset_reset="earliest",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    )

```

```

await consumer.start()
logger.info("[CONSUMER] Подключен к Kafka, ожидаю сообщения...")

try:
    async for msg in consumer:
        logger.info(
            f"[CONSUMER] Обработка: partition={msg.partition}, "
            f"offset={msg.offset}, value={msg.value}"
        )

        # Здесь может быть любая бизнес-логика:
        # - запись в БД
        # - обновление кэша
        # - вызов внешнего API
        await handle_event(msg.value)

except asyncio.CancelledError:
    logger.info("[CONSUMER] Получен сигнал остановки.")
    raise
finally:
    await consumer.stop()
    logger.info("[CONSUMER] Отключен.")

async def handle_event(event: dict):
    """
    Обработка одного события.
    """
    action = event.get("action")
    user_id = event.get("user_id")

    # Имитация работы
    await asyncio.sleep(0.5)

    logger.info(f"[HANDLER] Событие '{action}' для user_id={user_id} обработан о.")

# =====
# ENDPOINTS API
# =====

@app.get("/")
def root():
    return {

```

```

        "service": "FastAPI + Kafka",
        "endpoints": {
            "publish": "POST /publish",
            "health": "GET /health",
        }
    }
}

```

```

@app.get("/health")
async def health():
    """Проверка, что Producer жив."""
    if producer is None:
        raise HTTPException(status_code=503, detail="Producer не инициализирован")
    return {"status": "ok", "kafka_connected": True}

@app.post("/publish", response_model=EventResponse)
async def publish_event(request: EventRequest):
    """
    Принимает событие от клиента и публикует его в Kafka.
    Возвращает метаданные: в какую партицию и под каким offset попало сообщение.
    """
    if producer is None:
        raise HTTPException(status_code=503, detail="Kafka Producer не готов")

    event_data = {
        "user_id": request.user_id,
        "action": request.action,
        "payload": request.payload or {},
        "source": "fastapi_api",
    }

    try:
        # Отправляем и ждём подтверждения
        result = await producer.send_and_wait(TOPIC_NAME, event_data)

        logger.info(
            f"[API] Опубликовано событие: topic={result.topic}, "
            f"partition={result.partition}, offset={result.offset}"
        )

    return EventResponse(
        status="published",
        topic=result.topic,
        partition=result.partition,

```

```
        offset=result.offset,  
    )  
  
    except Exception as e:  
        logger.error(f"[API] Ошибка публикации: {e}")  
        raise HTTPException(status_code=500, detail=f"Ошибка Kafka: {str(e)}")
```

Шаг 4. Запуск

```
In [17]: cd ~/docker-module11  
         source venv/bin/activate  
         uvicorn main:app --reload --port 8000
```

Что произойдёт при старте:

1. FastAPI вызовет `lifespan`.
2. Producer подключится к Kafka.
3. Запустится фоновая задача `kafka_consumer_loop`.
4. API начнёт слушать порт 8000.

Тестирование:

```
In [18]: # Проверка здоровья  
         curl http://localhost:8000/health  
  
         # Отправка события  
         curl -X POST "http://localhost:8000/publish" \  
             -H "Content-Type: application/json" \  
             -d '{"user_id": 42, "action": "model_training_started", "payload": {"model": "xgboost"}}'  
  
         # Отправим ещё несколько  
         curl -X POST "http://localhost:8000/publish" \  
             -H "Content-Type: application/json" \  
             -d '{"user_id": 7, "action": "dataset_uploaded", "payload": {"size_mb": 150}}'
```

Наблюдайте в терминале с uvicorn:

- API мгновенно отвечает (Producer работает асинхронно).
- Фоновый Consumer получает сообщения и обрабатывает их.
- Вы видите логи и от API, и от Consumer'a в одном терминале.

11.6. Docker Compose: API + Kafka вместе

Если хотите запустить всё в Docker (как в Модуле 5), вот `docker-compose.yml` :

```
In [19]: services:
          kafka:
            image: docker.io/bitnami/kafka:3.7
            container_name: kafka_broker
            ports:
              - "9092:9092"
            environment:
              - KAFKA_CFG_NODE_ID=0
              - KAFKA_CFG_PROCESS_ROLES=controller,broker
              - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
              - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
              - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
              - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
              - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=0@kafka:9093
              - KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=true

          api:
            build: .
            ports:
              - "8000:8000"
            environment:
              - KAFKA_BOOTSTRAP_SERVERS=kafka:9092
            depends_on:
              - kafka
```

Важное изменение в коде: Если API работает внутри Docker, адрес Kafka меняется с `localhost:9092` на `kafka:9092` (имя сервиса). Используйте переменную окружения:

```
In [20]: import os
          KAFKA_BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")
```

11.7. Обработка ошибок и надёжность

11.7.1. Что если Kafka недоступна при старте?

В `lifespan` можно добавить `retry`:


```
In [21]: import aiokafka.errors

async def start_producer_with_retry():
    for attempt in range(10):
        try:
            await producer.start()
            return
        except aiokafka.errors.KafkaConnectionError:
            logger.warning(f"Kafka недоступна, попытка {attempt + 1}/10...")
            await asyncio.sleep(2)
    raise Exception("Не удалось подключиться к Kafka")
```

11.7.2. Что если отправка падает?

`send_and_wait` выбросит исключение. В FastAPI endpoint мы ловим его и возвращаем HTTP 500. В production можно:

- Записать в локальную очередь и повторить позже.
- Использовать Circuit Breaker (не отправлять в Kafka, пока она недоступна).
- Вернуть клиенту ошибку, чтобы он повторил запрос.

11.8. Итоги модуля: чек-лист

- ☐ Понимаю, зачем нужен **асинхронный** клиент: чтобы не блокировать API на время сетевого обмена с Kafka.
- ☐ Знаю разницу между синхронным (`kafka-python`) и асинхронным (`aiokafka`) клиентом.
- ☐ Умею устанавливать `aiokafka` .
- ☐ Знаю, что `async def` + `await` позволяют «отпускать» управление во время ожидания.
- ☐ Умею создавать `AIOKafkaProducer` с `value_serializer` .
- ☐ Понимаю разницу `send()` (быстро, без гарантии) и `send_and_wait()` (надёжно, с подтверждением).
- ☐ Умею получать `RecordMetadata` (topic, partition, offset) после отправки.
- ☐ Умею создавать `AIOKafkaConsumer` с `value_deserializer` и `group_id` .
- ☐ Знаю, что `async for msg in consumer` — это неблокирующий цикл чтения.
- ☐ Понимаю, что такое `auto_offset_reset="earliest"` vs `"latest"` .
- ☐ Знаю про автокоммит offset'ов и умею отключать его (`enable_auto_commit=False`) для ручного контроля.
- ☐ Умею делать ручной `await consumer.commit()` .

- ☐ Интегрировал aiokafka в FastAPI через `lifespan` .
- ☐ Знаю, как запустить фоновую корутину-консьюмер через `asyncio.create_task()` .
- ☐ Умею корректно останавливать Producer и Consumer при завершении приложения.
- ☐ Понимаю, как адаптировать код для работы внутри Docker Compose (имена сервисов вместо localhost).

В следующем модуле мы поднимемся на финальный уровень: научимся **масштабировать воркеров**, обрабатывать сбои через **ребалансировку** Consumer Groups и реализовывать **идемпотентность** — ключевой паттерн для систем с гарантией at-least-once.